Project Design Phase – Solution Architecture

Parameter	Details
Date	14 February 2026
Team ID	LTVIP2026TMIDS79181
Project Name	Smart Sorting – Rotten Fruit Detection using Machine Learning
Maximum Marks	4 Marks

Overview

The solution architecture bridges the business goal of reducing food waste and improving fruit quality detection with the technical implementation using image preprocessing, deep learning models, and a web-based interface. The architecture defines system components, data flow, interfaces, and deployment considerations for the Smart Sorting system.

Key Goals

• Provide accurate detection of fresh and rotten fruits using image classification
• Reduce manual effort in fruit sorting and quality inspection
• Ensure modular architecture across image preprocessing, model prediction, and web interface layers
• Enable scalability for larger datasets and future agricultural applications
• Maintain reproducibility using saved trained models and preprocessing configurations

Architecture Components

Frontend (UI):
Web interface where users upload fruit images and view prediction results.

Backend (API Layer):
Flask-based server that handles image uploads, input validation, and prediction requests.

Preprocessing Layer:
Image preprocessing including resizing, normalization, and augmentation to prepare images for the machine learning model.

ML Model Layer:
Trained deep learning model (CNN / Transfer Learning model) that classifies fruit images as **Fresh or Rotten**.

Model Storage:
Saved trained model file (e.g., `.h5` or `.keras`) used for inference during prediction.

Prediction Output Layer:
Displays classification result (Fresh / Rotten) along with prediction confidence.

Data Flow

1. User uploads a fruit image through the web application.
2. Backend server receives the image and validates the input format.
3. Image is processed through preprocessing steps (resize, normalization).
4. Preprocessed image is passed to the trained deep learning model.
5. Model predicts whether the fruit is **Fresh or Rotten**.
6. Prediction result is displayed on the web interface for the user.

Solution Architecture Diagram

User → Web Interface → Flask Backend → Image Preprocessing → ML Model (CNN) → Prediction Result → Display Output

Non-Functional Considerations

• **Performance:** Fast image prediction (<1 second per image)
• **Reliability:** Consistent preprocessing pipeline ensures accurate predictions
• **Security:** Input validation for uploaded images
• **Maintainability:** Modular code structure for model, preprocessing, and UI
• **Scalability:** Can be extended for multiple fruit types and larger datasets
• **Deployability:** Can be deployed on cloud platforms or integrated with automated fruit sorting systems